EDAV Self Assessment

Date:20/04/2026

Data Distribution Visualization with IrisDataset

1. Density Plots (KDE - Kernel Density Estimate)

What they are: Density plots visualize the distribution of one or more numerical variables. They estimate the Probability Density Function (PDF) of a continuous variable. Instead of using bars like a histogram, they use a continuous curve to show where data points are concentrated.

Types and Examples :

- **Univariate Density Plot (Single Variable):** Plot 1 (sepal_length)
  - **Purpose:** Shows the distribution shape (e.g., unimodal, bimodal, skewed), central tendency, and spread of a single variable.
  - **Example:** In Plot 1, the curve for sepal_length shows its most frequent values and how values taper off.
- **Overlaid Density Plots (Comparison by Category):** Plot 2 (petal_length by species)
  - **Purpose:** Compares the distributions of a single numerical variable across different categories.
  - **Example:** In Plot 2, we see distinct petal_length distributions for 'setosa', 'versicolor', and 'virginica' species, highlighting how they differ in this feature.

2. Contour Plots (2D Density Plots)

What they are: Contour plots are used to visualize the joint distribution of two numerical variables. They show regions where combinations of these two variables are more or less dense. Think of them like topographic maps, where lines (or filled areas) connect points of equal data density.

Types and Examples :

- **2D Filled Contour Plot:** Plot 3 (petal_length vs. petal_width)
  - **Purpose:** Displays the joint density of two variables using shaded areas. Warmer colors typically indicate higher data concentration.
  - **Example:** In Plot 3, the vivid yellow/green areas show the combinations of petal_length and petal_width that are most common in the Iris dataset.
- **2D Contour Plot (Lines Only):** Plot 4 (petal_length vs. petal_width)
  - **Purpose:** Uses distinct lines to represent specific levels of data density. This is useful for precisely identifying boundaries of high-density regions.
  - **Example:** In Plot 4, the lines show different thresholds of combined petal_length and petal_width frequency, making it easy to see the 'ridges' of data concentration.

Key Differences and When to Use Them:

Fea ure	Density Plot (KDE)	Contour Plot (2D Density)
Number of Variables	Primarily 1 numerical variable (can be overlaid for categorical comparisons)	Exactly 2 numerical variables
Dimensions	1D distribution (or 1D distribution for each category)	2D joint distribution
Output	A curve (or multiple curves) showing probability density	Lines or filled areas connecting points of equal density in a 2D plane
Purpose	Understand individual variable distribution, detect skewness, modes, compare distributions across groups.	Understand the relationship and joint distribution of two variables, identify clusters, correlations, and high-frequency regions.
Best for	Showing frequency of single variable values.	Showing where pairs of variable values occur most frequently.

In summary:

- Use **Density Plots** to understand the characteristics of *individual* features.
- Use **Contour Plots** to understand the *relationship and co-occurrence* of *two* features.

Setup and Data Loading: Iris Dataset

This performs the initial setup by importing necessary libraries and loading the well-known Iris dataset.
import seaborn as sns: Imports the Seaborn library, which is built on Matplotlib and provides a high-level interface for drawing attractive statistical graphics.
import matplotlib.pyplot as plt: Imports Matplotlib's pyplot module, which provides a MATLAB-like interface for creating plots and visualizations.
ris = sns.load_dataset('iris'): Loads the Iris dataset directly from Seaborn's online data repository. The Iris dataset is a classic and widely used multivariate dataset in machine learning and statistics, often used for classification and clustering examples. It contains 150 samples of iris flowers with four features (sepal length, sepal width, petal length, and petal width) and their corresponding species.
print("Iris Dataset Head:"): Prints a descriptive header.
print(iris.head()): Displays the first 5 rows of the `iris` DataFrame. This gives a quick overview of the data structure, column names, and the types of values present.
print("\nIris Dataset Description:"): Prints another descriptive header.**print(iris.describe()):** Generates descriptive statistics of the numerical columns in the `iris` DataFrame, including count, mean, standard deviation, min, max, and quartiles. This helps in understanding the central tendency, dispersion, and shape of the dataset's distribution.

```
import seaborn as sns
import matplotlib.pyplot as plt
iris = sns.load_dataset('iris')
print("Iris Dataset Head:")
print(iris.head())
print("\nIris Dataset Description:")
print(iris.describe())
```

OUTPUT:

Iris Dataset Head: sepal_length sepal_width petal_length petal_width species 0 5.1 3.5 1.4 0.2 setosa 1 4.9 3.0 1.4 0.2 setosa 2 4.7 3.2 1.3 0.2 setosa 3 4.6 3.1 1.5 0.2 setosa 4 5.0 3.6 1.4 0.2 setosa Iris Dataset Description: sepal_length sepal_width petal_length petal_width count 150.000000 150.000000 150.000000 150.000000 mean 5.843333 3.057333 3.758000 1.199333 std 0.828066 0.435866 1.765298 0.762238 min 4.300000 2.000000 1.000000 0.100000 25% 5.100000 2.800000 1.600000 0.300000 50% 5.800000 3.000000 4.350000 1.300000 75% 6.400000 3.300000 5.100000 1.800000 max 7.900000 4.400000 6.900000 2.500000

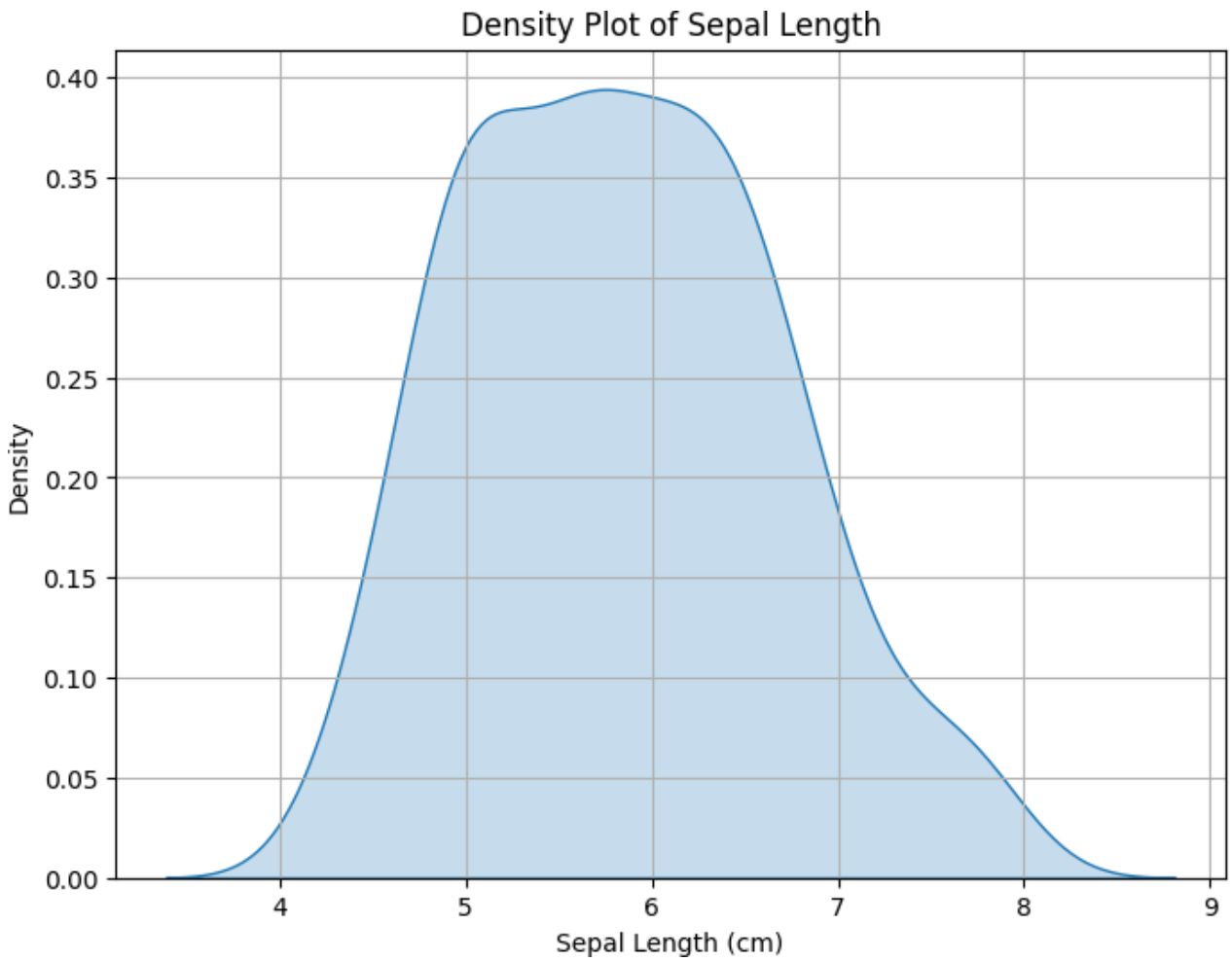
Plot 1: Density Plot for a Single Numerical Variable

This plot uses a Kernel Density Estimate (KDE) to visualize the distribution of a single numerical variable, `sepal_length`. A KDE plot estimates the probability density function of a random variable. The x-axis represents the range of the `sepal_length` values, and the y-axis represents the density. The shaded area under the curve indicates the likelihood of observing a particular value. For instance, a higher density peak suggests that `sepal_length` values around that point are more common. This plot is useful for understanding the overall shape of the data's distribution, identifying modes (peaks), and assessing skewness or kurtosis. The `fill=True` argument makes the area under the curve shaded, which can improve readability.

CODE:

```
plt.figure(figsize=(8, 6))
sns.kdeplot(data=iris, x='sepal_length', fill=True)
plt.title('Density Plot of Sepal Length')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Density')
plt.grid(True)
plt.show()
```

OUTPUT:



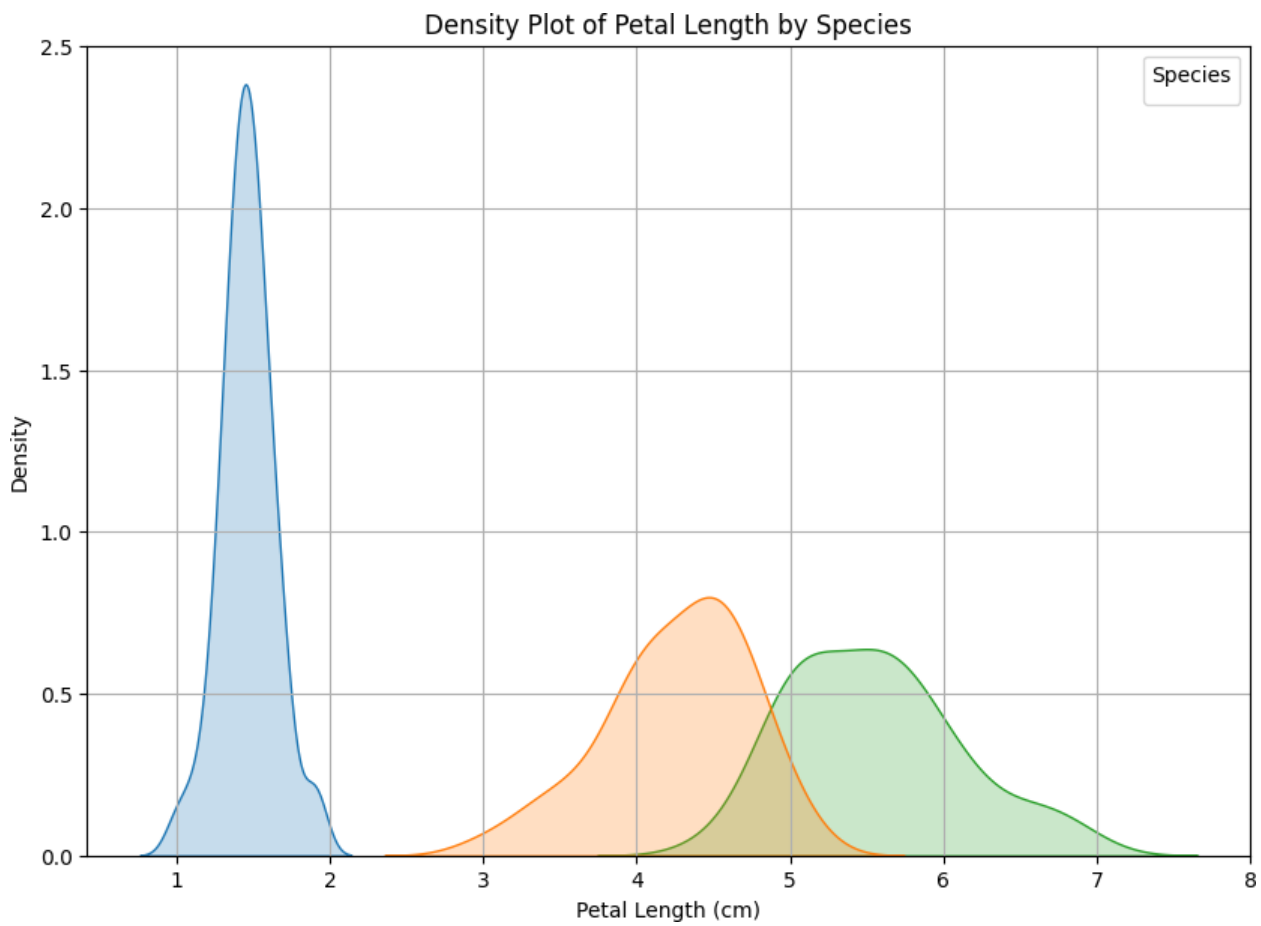
Plot 2: Multiple Density Plots for Comparison by Category

This visualization overlays multiple Kernel Density Estimates (KDEs) on a single graph, allowing for a direct comparison of the distribution of a numerical variable (petal_length) across different categories (species). Each colored line and shaded area represents the density distribution for one species. By examining the positions, shapes, and overlaps of these curves, we can easily see how petal_length differs (or is similar) among the 'setosa', 'versicolor', and 'virginica' iris species. The hue='species' argument tells Seaborn to create a separate density curve for each unique value in the 'species' column, while fill=True shades these areas for better visual distinction. common_norm=False ensures that each density curve is normalized independently, making their individual shapes comparable even if the group sizes are different.

CODE:

```
plt.figure(figsize=(10, 7))
sns.kdeplot(data=iris, x='petal_length', hue='species', fill=True, common_norm=False)
plt.title('Density Plot of Petal Length by Species')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Density')
plt.legend(title='Species')
plt.grid(True)
plt.show()
```

OUTPUT:



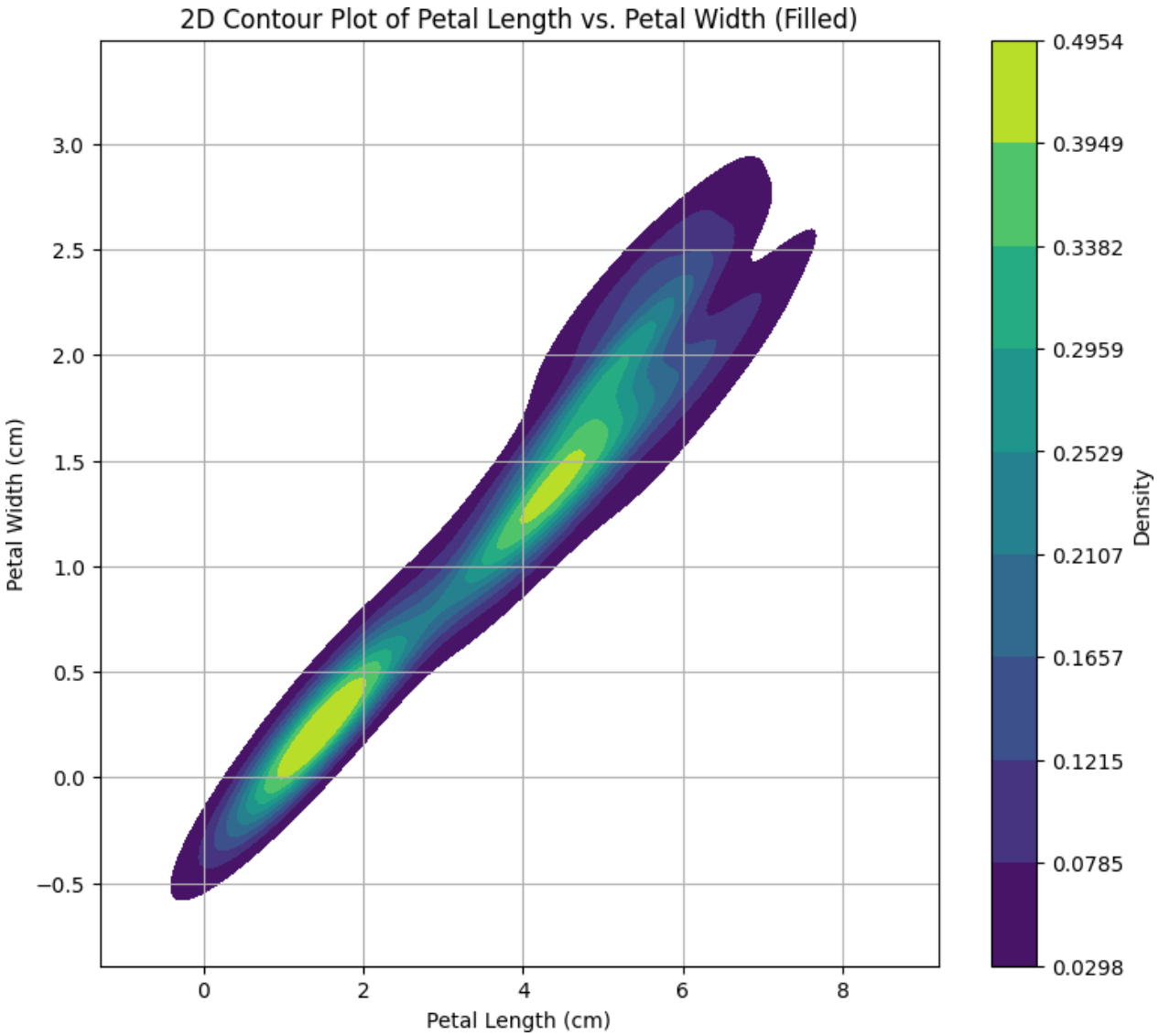
Plot 3: 2D Filled Contour Plot for Joint Distribution

This plot visualizes the joint probability density of two numerical variables, petal_length and petal_width, using a 2D filled contour map. Unlike 1D density plots that show the distribution of a single variable, this plot illustrates where combinations of petal_length and petal_width are most frequent. The contours connect points of equal density, and the fill=True argument shades the areas between these contours with different colors. Typically, warmer colors (like those in 'viridis' colormap) indicate regions of higher data concentration (more frequent combinations of length and width), while cooler colors represent lower densities. This plot is excellent for identifying clusters, correlations, and general patterns in the relationship between two continuous variables. The levels argument controls the number of contour lines drawn, influencing the granularity of the density representation.

CODE:

```
plt.figure(figsize=(9, 8))

ax = sns.kdeplot(data=iris, x='petal_length', y='petal_width', fill=True, cmap='viridis', levels=10)

plt.title('2D Contour Plot of Petal Length vs. Petal Width (Filled)')

plt.xlabel('Petal Length (cm)')

plt.ylabel('Petal Width (cm)')

plt.colorbar(ax.collections[0], label='Density')

plt.grid(True)

plt.show()
```

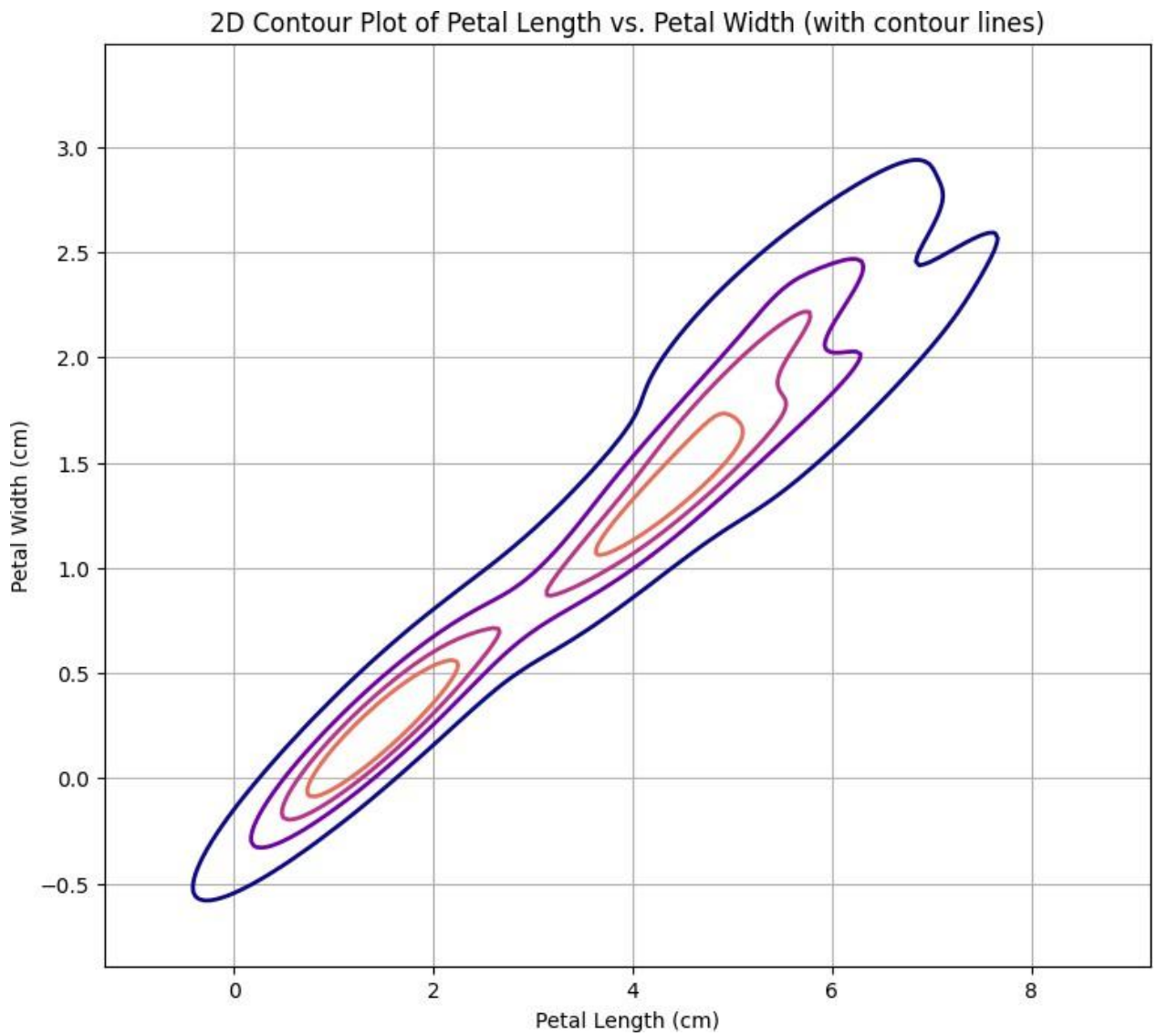
OUTPUT:



Plot 4: 2D Contour Plot with Highlighted High-Density Regions

Similar to the previous 2D contour plot, this visualization also explores the joint distribution of petal_length and petal_width. However, instead of filling the areas, it uses discrete contour lines to represent different levels of data density. Each line connects points where the density of data occurrences is equal. This type of plot is particularly useful for discerning the shape and extent of high-density regions and potential clusters without the visual clutter of filled areas. The levels argument directly controls the number of contour lines. Using a colormap like 'plasma' helps to differentiate between these density levels visually, and linewidths enhances the visibility of the contour lines. This plot helps in understanding the correlation between the two variables and where the majority of observations lie in their combined space.

CODE:

```
plt.figure(figsize=(9, 8))
sns.kdeplot(data=iris, x='petal_length', y='petal_width', cmap='plasma', levels=5, linewidths=2)
plt.title('2D Contour Plot of Petal Length vs. Petal Width (with contour lines)')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.grid(True)
plt.show()
```

OUTPUT: